Logbook — Anomalous transport in soaring flights

Matteo Di Sante

Working log

Working notebook. The timeline is regenerated from the git history; the narrative notes are written by hand.

1 Timeline (auto-generated from git)

Date	Event (commit subject)
2026-07-06	ci(docs): auto-deploy on push again, scoped to paths that feed the site
2026-07-06	docs(logbook): reformat narrative notes for readability; re-sync Next steps; docs deploy on demand
2026-07-06	fix(altitude-noise): dual-axis + decoupled windows for Fig. 4.1 panels (a)/(b); chore(deps): dev/analysis as uv default groups
2026-07-06	docs: data-driven fix-level section; four-panel altnoise caption; blueprint, logbook, README
2026-07-06	chore(data): explicit SSD->repo seasons_index sync; document the offline snapshot
2026-07-06	feat(altitude-noise): four-panel figure (raw channels, difference, PSD, availability)
2026-07-06	feat(preproc): data-driven fix-level diagnostics; gap-histogram readability; drop redundant time-gap
2026-07-06	fix(igc): treat only full-day drops as a midnight roll-over
2026-07-06	chore: SSD data layout (raw/catalog/derived) + pyarrow for the scan cache
2026-07-05	thesis ch.4: data-driven pre-processing pipeline + geodesy figures
2026-07-02	Symmetric para/hang dataset; adopt barometric altitude; IGC parser + noise diagnostics
2026-07-01	Rename soaring-ffvl -> soaring-para; add hang-glider data and update all docs
2026-07-01	Fix delta config: season start 2001, data_root /Volumes/SSD_DISANTE/hang_gliders/delta_cfd_igc
2026-07-01	Add soaring-delta CLI for hang-glider CFD data (delta.ffvl.fr)
2026-07-01	Add analysis sub-package and preprocessing diagnostics; revise thesis chapters 1-4
2026-07-01	Expand pre-processing: explicit cleaning and filtering sections
2026-07-01	Clarify stationarity/compatibility check paragraph
2026-07-01	Add preliminary characterization section before main observables
2026-07-01	Add isotropy test protocol to the propagator item
2026-07-01	Rewrite propagator item: formal derivation of marginal scaling
2026-07-01	Reorganize TA-MSD item for clarity and typographic structure
2026-07-01	Clarify ergodicity breaking and add CTRW appendix
2026-06-28	Revise chapter 4 observables and the IGC description

Date	Event (commit subject)
2026-06-28	Warn when thesis and logbook next-steps drift apart
2026-06-28	Align logbook next steps with the thesis planning chapter
2026-06-28	Publish the logbook alongside the thesis
2026-06-28	Add living thesis document and build automation
2026-06-26	Deploy docs via GitHub Pages Actions artifact instead of gh-pages branch
2026-06-26	Fail fast with a clear message when data_root is unset
2026-06-26	Add 'verify' subcommand and remove machine-specific data_root path
2026-06-26	Link published docs in README; drop Development section
2026-06-25	Add GitHub Pages deployment via gh-pages branch
2026-06-25	Translate all documentation, comments, and strings to English
2026-06-25	Initialize thesis repo: FFVL CFD .igc data acquisition

2 Narrative notes

2026-06 — Paraglider data acquisition complete. The full FFVL paraglider CFD archive (`parapente.ffvl.fr`, seasons 1999–2000 to 2025–2026) has been scraped: about 203,000 flight records, of which ∼186,000 carry a GPS track. All 186,052 available tracks were downloaded.

What worked Impersonating a Chrome TLS fingerprint (`curl_cffi`) reliably passes the FFVL Cloudflare challenge; the resumable, atomic, validated download survived many interruptions without corruption.

Gotchas 173 tracks could not be obtained — 172 because the server returns something that is not a valid IGC (broken or placeholder links on FFVL's side, so a retry does not help), and 1 due to a transient Cloudflare challenge (worth a later retry). On macOS/exFAT the OS scatters `._*` sidecar files next to every written file; a `clean` step removes them.

2026-06 — Integrity verified. A full on-disk re-scan (`verify`) confirmed that all 186,052 downloaded files are structurally valid IGC: no truncated files, no leftover `.part`, no read errors. The verification logic was promoted from a throwaway script to a first-class `soaring-para verify` subcommand (reusing the download-time validator), with tests.

2026-06 — Robustness of the data path. The external disk remounted under a different name (`HDD_DISANTE` → `SSD_DISANTE`), which would have silently broken a hard-coded path. Fix: removed every machine-specific path from the repo (placeholder + `SOARING_PARA_DATA_ROOT` environment variable) and added a start-up guard that aborts with a clear, actionable message when `data_root` is unset/unmounted — and verified that `/Volumes` is not user-writable, so a wrong path can never cause a silent download to the wrong place.

2026-07 — Hang-glider data acquired; naming corrected.

Second source added The hang-glider CFD (`delta.ffvl.fr`, seasons 2001–2002 to 2025–2026), sharing the acquisition code (`soaring.acquisition.ffvl`) via a second CLI entry point `soaring-delta` (config `delta_download.yaml`, env var `SOARING_DELTA_DATA_ROOT`). Data: ∼9,300 flights, 6,716 tracks downloaded, 30 failures (broken server-side links, same pattern as paragliders). Stored at `/Volumes/SSD_DISANTE/hang_gliders/delta_cfd_igc/`.

Naming corrected The paraglider CLI was renamed from the generic `soaring-ffvl` to `soaring-para` (and `SOARING_FFVL_DATA_ROOT` → `SOARING_PARA_DATA_ROOT`), and its config from `ffvl_download.yaml` to `para_download.yaml`, to avoid ambiguity now that both sources are present.

2026-07 — Hang-glider catalog built; datasets reorganized symmetrically. Ran the full pipeline on the hang-glider data (`soaring-delta clean`, `verify`, `build-catalog`, `status`), which so far had only been downloaded: 9,259 flights, 6,746 with a track, 6,716 downloaded and all verified (0 integrity failures), 30 unrecoverable server-side links.

Data layout The local `data/` folder now mirrors the SSD — one directory per discipline (`paragliders/`, `hang_gliders/`), each with a versioned `seasons_index.csv` and a git-ignored `catalog.csv`.

Thesis symmetry `generate_stats.py` now reads both indices and emits per-discipline macros (`\StatPara*`, `\StatHang*`) and two per-season tables; the first three thesis chapters were made symmetric across both disciplines.

2026-07 — Altitude channel decision: barometric. Decided to take the vertical dynamics from the *barometric* altitude rather than the GNSS altitude, diverging from the reference pipeline (Jérémie's code derives z and v_z from `AltGNSS`, reading but not using `AltPre`).

Rationale The pressure altitude has sub-metre relative precision and low high-frequency noise, its errors being slow (offset, drift) and killed by differencing, whereas the GNSS vertical carries a high-frequency noise floor that derivatives amplify. Confirmed empirically with a new noise diagnostic — an IGC `B`-record parser (`soaring.analysis.igc`) and an altitude-noise analysis (`soaring.analysis.altitude_noise`) whose Welch power spectra show the GNSS floor two to three orders of magnitude above the barometric one near Nyquist (thesis appendix on the method).

Consequences The `A`/`V` validity flag is subsumed by the missing-altitude check on the adopted channel (a `V` fix keeps position + baro on a baro flight, but is dropped on a GNSS-fallback flight); one altitude channel per trajectory, no baro/GNSS splicing; flights with no pressure sensor fall back to unfiltered GNSS, tagged by a per-flight `alt_source` column; the trimming threshold is on horizontal speed only ($v_{xy} > 5\,\mathrm{m/s}$, sustained); cleaning/trimming run on the raw geographic coordinates (great-circle speeds), with the ENU conversion done *last*, origin at take-off.

2026-07 — Baro-absence fraction: sized sampling, not a full sweep. A full-population parse of the paraglider archive (186,052 files) for the baro-channel-absence figure took an unacceptable ∼45 min serially; parallelising across 8 processes cut this to ∼15 min, still too slow to run routinely.

Fix A properly justified *simple random sample*: the standard proportion sample-size formula ($n = z^2 p(1-p)/e^2$, conservative $p = 0.5$) gives $n = 2401$ for a target 95%-confidence margin of error of ±2 percentage points (`required_sample_size` in `soaring.analysis.altitude_noise`, with `proportion_ci` reporting the point estimate and interval). The population is treated as effectively infinite: at $n = 2401$ out of 186,052 flights, the finite-population correction would move n by only ∼30, not worth the extra parameter. The hang-glider archive (6716 files) stays a full census (cheap enough, under two minutes). Total runtime: ∼50 s. Result: paragliders $(28.4 \pm 1.8)\,\%$ (95% CI, sampled), hang gliders $17.5\,\%$ (exact, census).

Also caught mid-flight The IGC parser accepted syntactically well-formed but semantically invalid `B` records (e.g. minute ≥ 60, hour ≥ 24, out-of-range latitude/longitude) without rejecting them; added explicit format-validity checks (9 new tests) and corrected the thesis wording, which had conflated this *implemented* format-validity layer with the fix-level *dynamics-plausibility* layer of Table 4.1 – at the time designed (a dataclass of thresholds) but not yet implemented as an enforced filter.

2026-07 — Pre-processing made data-driven and config-driven. Reworked Chapter 4 so the flight-level filtering is computed from the *full parsed dataset* (paragliders and hang gliders),

not the declared catalog metadata.

What changed A track scan (`soaring.analysis.preprocessing.scan_tracks`) parses every downloaded track; Figure 4.2 shows the real duration and flown-path-length distributions with per-variable *marginal* retention curves. Adopted cuts: duration ≥ 40 min and path length ≥ 30 km (a minimal cut, $\sim$95% retained); the earlier min-fix-count cut was dropped as redundant with duration, and the *extent* proxy was replaced by total flown *path length* (median 86 km para, 158 km hang).

Key decision Every pre-processing threshold now lives in `configs/preprocessing.yaml` (never hard-coded), loaded via `load_preproc_config` into typed dataclasses.

Also added Great-circle (haversine) inter-fix speed/distance *before* the ENU conversion; a Savitzky–Golay smoothing/differentiation section with a noise-matched procedure for its two hyperparameters; the switch from a common resampling cadence to native per-flight Δt (Figure 4.5); and an implementation blueprint (`docs/guide/preprocessing-pipeline.md`) recording the exact schema and steps, to retire into code once the pipeline is built.

2026-07 — Gap diagnostics; scan cache; baro-presence consolidated. Added Figure 4.5, the sampling-regularity diagnostics (largest gap relative to native Δt, missing fraction of a uniform grid, each with its marginal retention curve), fulfilling a promise Sec. 4.1.6 already made but had not yet delivered.

Scan cache A fresh full parse would have repeated the $\sim$15–20 min paraglider census just done for Figure 4.2, so the per-flight scan is now **cached** to `data/<discipline>/track_scan.parquet` (gitignored, like `catalog.csv`; `load_or_scan_tracks` reads it if present, else scans and writes it — later moved to the SSD, see below). `track_stats` was extended with a few near-free byproducts of the same scan — barometric-presence fraction, max horizontal/vertical speed, barometric altitude range — for future validation of Table 4.1's fix-level bounds.

Consolidation The barometric-presence fraction (Sec. 4.1.1) now reads from this same cache (`altitude_noise.baro_presence_from_scan`) instead of running its own separate scan, turning the paraglider number from a sized sample into an exact census at no extra cost: 30.1 % (exact, $n = 186{,}025$) — consistent with the earlier sampled estimate, (28.4 ± 1.8) %.

Also caught A pre-existing fragility in `altitude_noise.collect` where a single unreadable file (a transient external-disk hiccup) aborted the whole PSD sample; it now skips and logs the file instead.

2026-07 — SSD reorganised; no data kept locally; figure fixes.

Policy going forward Raw data, catalogs and every analysis byproduct live only on the external SSD, never in the local repo.

Reorganisation Each discipline's `data_root` is now laid out by maturity — `raw/{igc,raw_xml}` (untouched acquisition output), `catalog/{catalog.csv,seasons_index.csv}` (tables derived from it), `derived/` (further byproducts, today just `track_scan.parquet`; future cleaned/filtered data and analysis results get their own `processed/`/`analysis/` siblings), with `Config` updated accordingly (new `derived_dir` property). Verified file counts before/after the move (paraglider: 186,052 tracks, 27 XMLs; hang glider: 6,716 tracks, 25 XMLs — all matched) and that both acquisition CLIs still work against the new layout. Previously-local `track_scan.parquet` copies moved to `<data_root>/derived/`; orphaned local `catalog.csv` copies (superseded by the track-based scan months ago) were removed.

Figure 4.5 (gaps) The adopted `max_gap_factor` was raised from 5 to 10 — at 5 it retained only 77.8 %/65.0 % of para/hang flights (nowhere near the near-full retention of the duration/path cuts), at 10 it reaches 86.9 %/85.1 %. Considered 20 (90.8 %/90.6 %, little further gain) and rejected it: the cut is *relative* to each flight's own native Δt, so for the slower hang-glider

loggers (up to 10 s native) a factor of 20 would tolerate interpolating across a ∼200 s gap — long enough to span a thermal climb. Also tightened the distribution panels' axis range to the data's own percentiles (the old 0–200/0–0.6 range was mostly empty).

Figure 4.1 (altitude noise) Panel (a)'s difference trace now uses a colour distinct from both channels (was reusing GNSS's red); its window grew from 10 to 30 minutes; a Nyquist-frequency line was added to panel (b) (previously discussed in the caption but never drawn). Also found and fixed a real bug in the "representative flight" selection: picking the *longest flight by elapsed time* broke on a 30-hour, 1,100-fix outlier (one corrupted timestamp gap) — fixed to select by *fix count* instead, which has no such failure mode.

Local data folder clarified The two versioned `seasons_index.csv` are the offline reproducibility anchor for the thesis's dataset counts (the document builds with no SSD mounted), not stray local data. Their sync from the canonical SSD copy is now explicit and automatic (`refresh_seasons_index.py`, run best-effort by the pre-commit hook), so the snapshot cannot silently drift.

2026-07 — Fix-level cleaning made data-driven; parser roll-over bug; figure and data-folder cleanups.

Fix-level cleaning audited Table 4.1 is now audited on the data, like the flight-level cuts: a new figure in Sec. 4.1.2 shows the per-*fix* distributions of horizontal speed, barometric vertical speed and barometric altitude (`make_fixlevel_diagnostics_figure`, from `fix_level_distributions` over a seeded sample — millions of fixes). What justifies a fix-level bound is the fraction of *fixes* it removes, not the number of flights it touches; the per-flight *maxima* already in the scan (`max_vxy_mps` and friends) answer the wrong question, so they do not place the cuts.

Gap handling unified Removed the absolute 60 s inter-fix time-gap from the fix-level table: it duplicated the sampling-regularity cut of Sec. 4.1.6, which is more principled (relative to each flight's own native Δt). Neither was ever an enforced filter, so this is a spec simplification, not a behaviour change.

Parser bug fixed Found the root cause of the "30-hour, 1,100-fix" pathology worked around previously: the IGC parser's midnight roll-over unwrap treated *any* backward step in time-of-day as a new day and added 86,400 s, so a few-second GPS glitch became a spurious multi-hour gap. Now only a drop of (nearly) a whole day counts as a roll-over, with residual backward jitter clamped to keep the series monotonic. Added regression tests for both the true roll-over and the glitch.

Figure and config cleanups The altitude-noise figure (Sec. 4.1.1) is now four panels — (a) raw channels, (b) their difference, (c) PSD, (d) fallback rate — fixing the old a/b/d labelling. The gap-diagnostics figure (Sec. 4.1.6) no longer clips its tail into an artificial edge spike, with an x-range fitted to the data's own percentiles. Result-specific numbers (retention percentages, one-off anecdotes) were pruned from YAML/code comments — such numbers belong in the thesis, not in reusable config.

Local data folder clarified Rewrote `data/README.md` and fixed several stale references to a flat `data/seasons_index.csv` path that never existed.

2026-07 — Coordinate/geodesy figures. Redrew the coordinate-transformation figures: Figure 4.3 (WGS84 ellipsoid; geodetic vs geocentric latitude) and Figure 4.4 (the ECEF→ENU rotation, in our notation), and added the ESA Navipedia citation for the ECEF conversion.

2026-07 — Altitude-noise figure (Fig. 4.1): panels (a)/(b) redesigned. A closer look at the rendered figure (not caught by any test, since nothing there errors) found two compounding problems in the one-flight time-series panels, both traced to a single shared 30-minute window: the

several-hundred-metre climb dominated the vertical scale, hiding the GNSS jitter the panel exists to show; and panel (b), subtracting only the scalar *mean* of the GNSS−barometric difference, ended up on a ±20 m scale that looked inconsistent with the ∼130 m gap plainly visible in panel (a).

First attempt (reverted) Shortened the shared window to 2 minutes and switched panel (b) to a *linear* detrend (matching `_welch`'s). This resolved both symptoms but destroyed something worth keeping: the GNSS−barometric difference actually drifts by tens of metres over a full flight (plausibly the barometric channel's departure from the ICAO standard atmosphere as altitude changes) — a real, physically meaningful feature that a 2-minute window is too short to show and a linear detrend actively removes.

Final design Decoupled the two panels' windows and mechanisms instead of forcing one compromise on both. Panel (a) keeps a short (2-minute) window but now plots barometric and GNSS on *two independent y-axes* (same numeric span, different baseline) rather than one shared axis, which previously had to stretch from one channel's minimum to the other's maximum — wasting on blank space exactly the room needed to compare the two channels' roughness; with matched spans the GNSS trace is visibly more jagged than the barometric one. Panel (b) reverts to the original 30-minute window with only the *mean* removed (no detrend), so the slow drift is visible again alongside the residual jitter, with the removed mean offset (139 m for this flight) annotated directly on the panel. Both panels' wording was generalised from "climb" to "vertical motion", since a short window can just as well land on a glide.

2026-07 — `dev/analysis` moved from pip extras to uv default dependency groups. Chasing the figure fix above, a bare `uv sync -reinstall-package soaring` (no `-extra` flags) silently reset the environment to the core-only dependency set, uninstalling `pytest`/`ruff`/`mypy`/`matplotlib`/`scipy`/`pyarrow` — and, on the way, hit a pre-existing `uv` bug (uninstall failing on packages whose `RECORD` file was missing) that required rebuilding `.venv` from scratch.

Root cause `dev` and `analysis` were plain PEP 621 `optional-dependencies`, so *any* `uv sync`/`uv run` invocation without matching `-extra` flags resyncs the venv down to exactly what was requested, silently dropping whichever extras the previous invocation had added — alternating flagged and unflagged commands across a session means constant install/uninstall churn.

Fix Moved both to `[dependency-groups]` (PEP 735) with `[tool.uv] default-groups = ["dev", "analysis"]`, so every `uv sync`/`uv run` includes them unconditionally. Neither group was ever meant to be pip-installed by an external consumer in practice (this repo is not published), so the loss of `pip install .[dev]`-style installability is immaterial; `docs` remains a real extra since it is genuinely on-demand. The docs-deploy CI workflow now opts out of the new default groups (`-no-default-groups`) to stay minimal.

Docs updated `README.md`, `docs/guide/installation.md` (dependency-groups table, pip fallback instructions), the two reporting scripts' now-unnecessary `uv run -with matplotlib -with scipy` incantations, and the `soaring.analysis` package docstring.

2026-07 — Logbook readability; Next-steps drift; docs deploy on demand. Three unrelated loose ends, tidied together.

Logbook typography The narrative notes below were dense, single-paragraph entries — hard to scan. Reformatted the longer ones into short lead sentences plus labelled sub-points (*what changed*, *why*, *gotchas*), and dropped a stale forward-reference in the altitude-channel entry ("to confirm on a larger sample once the SSD is remounted", superseded two entries later by an exact 186,025-flight census).

Next-steps drift The Next steps list below had fallen out of step with the thesis chapter it is meant to mirror: *Preliminary characterization* (thesis Sec. 4.2) and *Tooling* (Sec. 4.9) were missing entirely, and the whole-trajectory, phase-resolved and transport-analysis bullets were missing content the thesis already has (the propagator/return-probability/first-passage observables; the step–waiting coupling and phase-sequence memory; the stratification and bias-control plan). Re-synced against the current chapter.

Docs deploy The GitHub Pages workflow redeployed on every push to `main`, including pushes with nothing to do with the site (e.g. acquisition/analysis code with no docstring change) — not wanted. One such deploy had also failed with a generic, transient "try again later" from GitHub's Pages API, but that was an isolated glitch, not a config problem (confirmed via the Actions log: only that one run failed out of the last ten, and only at the deploy step, after a successful build). Fix: the `push` trigger is now scoped with `paths` to what actually feeds the built site (`docs/**`, `mkdocs.yml`, `src/soaring/**`, whose docstrings mkdocstrings renders into the API reference), so the site still updates itself automatically but only for changes that matter to it; `workflow_dispatch` stays available for an on-demand redeploy.

2026-07 — License added; README badges; a real test-running CI. `pyproject.toml` had declared `license = MIT` since the project's start, but no `LICENSE` file existed, and the repo had no workflow that actually ran the test suite (only the docs-deploy one).

License Added `LICENSE` (MIT): permissive, minimal text, the de facto standard for research Python code, and consistent with what `pyproject.toml` already declared – no reason to pick something more restrictive for code meant to be freely reused.

New CI: `tests.yml` Runs `uv run pytest` with coverage on every push to `main` and every PR. Needed no `-extra` flags to get `pytest-cov/matplotlib/scipy/pyarrow` – a direct payoff of making `dev/analysis` uv default groups earlier. Coverage XML uploads to Codecov as a best-effort step (`fail_ci_if_error: false`): a Codecov hiccup, or the repo not being linked there yet, must never redden the Tests badge.

README badges Added five badges (python version, Tests workflow status, Codecov coverage, docs-online link, license) at the top of `README.md`. The Codecov one needs a one-time manual step outside this repo – linking `matteodisante/soaring-anomalous-transport` on `codecov.io` (GitHub login) and, if it asks for one, adding `CODECOV_TOKEN` as a repo secret – which cannot be done from here.

2026-06 — Documentation. Project docs are built with MkDocs and published on GitHub Pages. Initially deployed with `mkdocs gh-deploy`, which accumulated one built-site commit per deploy on a `gh-pages` branch; migrated to the official GitHub Pages *artifact* workflow (build → upload artifact → deploy), removing the branch and keeping the git history clean.

Operational note. In this repo `uv sync` occasionally fails to (re)install the editable package; `uv sync -reinstall-package soaring` fixes it.

3 Next steps

These mirror the planning chapter of the state document (`thesis/sections/04-next-steps`, itself titled "Next steps") and are kept in sync with it, section by section. The near-term plan is to analyse the data *in parallel* with studying the segmentation, since many observables can be built before any phase splitting.

- **Data sources.** FFVL paraglider and hang-glider CFD data are collected, cataloged and verified. Extend to further sources – notably WeGlide for sailplanes (a higher-quota API

request is pending; the default rate limit makes a bulk export infeasible) and possibly XContest – merging into one catalog and stratifying by source. Refresh the per-discipline `seasons_index.csv` via `build-catalog` so the document statistics stay current.

- **Pre-processing (thesis Sec. 4.1).** The parser is in place (`soaring.analysis.igc`) and the pipeline is designed, thresholded (`configs/preprocessing.yaml`) and audited on real data: fix-level cleaning (great-circle speeds), v_{xy}-based trimming, data-driven flight-level filtering (duration + path length), the ENU conversion (barometric vertical), native-Δt uniformity, and Savitzky–Golay smoothing/differentiation. Next: implement it end-to-end as production code (parser $\to \ldots \to$ Parquet `fixes/flights_meta` tables) per the blueprint, and measure the real ENU power spectra to finalize the two Savitzky–Golay smoothing timescales (still placeholders in the config).
- **Preliminary characterization (Sec. 4.2).** Lightweight, segmentation-free checks before the main analysis: per-flight trajectory statistics (duration, path length, fix count, native Δt); an isotropy check (the per-component variance ratio $\langle E(t)^2\rangle/\langle N(t)^2\rangle$, expected identically 1, plus a Kolmogorov–Smirnov test) needed before the marginal-propagator scaling can be used; compatibility across strata (glider class, season) via overlaid per-component displacement-variance curves, to justify – or rule out – pooling them; and a spatial-coverage check (take-off and displacement maps) for possible geographic stratification.
- **Whole-trajectory observables (Sec. 4.3, no segmentation).** Characterise the data with observables that need no phases: the MSD and its scaling exponent α (the primary anomalous-transport test); the time-averaged MSD, whose agreement or disagreement with the ensemble MSD is an ergodicity test; the one-dimensional marginal propagator and its scaling collapse; the return probability and the non-Gaussian parameter; the velocity autocorrelation and turning-angle distribution; first-passage/first-exit times; the speed/vertical-velocity distributions; and geometric measures (tortuosity, fractal dimension, radius of gyration), read both as one number per flight and as functions of scale. Goal: a first physical/statistical picture, independent of any phase splitting.
- **Segmentation (Sec. 4.4).** Study Jérémie's internship report and code, then re-examine the transition/search/climb HMM (binary features: sign of vertical speed, curvature-angle persistence, straightness) — decide whether to keep that approach or add complexity for a better segmentation on our larger, more heterogeneous data.
- **Phase-resolved observables (Sec. 4.5).** After segmentation: the waiting-time decomposition (climb τ_c, search τ_s, and their sum τ_w), the glide step-length/duration joint law, per-flight anatomy (thermal count, phase time/distance budget), and the angular diffusion between successive glides. Plus two couplings that decide which transport model applies: the step–waiting coupling (ℓ vs τ_w, the Lévy-walk signature) and memory in the phase sequence (transition-matrix structure, duration correlations), which test the renewal assumption behind the textbook CTRW/Lévy-walk results.
- **Reproduce the reference (Sec. 4.6).** Check that the enlarged paraglider-and-hang-glider dataset reproduces Vilpellet et al.: per-phase conditional MSD (their Fig. 3) and per-phase descriptive statistics (their Fig. 4), plus the global $H \approx 0.88$ — validation and universality test (the sailplane comparison becomes reachable once that source is in hand).
- **Transport analysis (Sec. 4.7).** Estimate the MSD scaling exponent, fit the tails of the waiting-time and step-length distributions with principled methods (e.g. maximum-likelihood power-law fits with goodness-of-fit tests), and perform CTRW-vs-Lévy-walk model selection using the step–waiting coupling. Results will be stratified (season, site, glider class, data source) and trajectories normalized, with finite-size, censoring and sampling biases controlled throughout.

- **Minimal model (Sec. 4.8).** A minimal stochastic model for $H \approx 0.88$: resolve transition segments, merge search+climb into one effective state, with angular diffusion between successive transition directions (preliminary idea, to be refined/changed).
- **Tooling (Sec. 4.9).** Largely underway already: the analysis package mirrors acquisition, and the dataset statistics plus several diagnostic figures (pre-processing, altitude noise) already regenerate automatically from the data rather than being pasted in by hand. Remaining: extend the same treatment to whatever tables/figures the transport analysis produces, once it exists.